

Ex:5 PACKET SNIFFER PROGRAM

AIM

To develop a packet sniffer using Python that captures and analyzes raw network packets, extracting Ethernet frame details such as source MAC, destination MAC, and protocol type.

ALGORITHM

1. **Initialize Host and Socket**
 - Retrieve the local IP address using `socket.gethostbyname(socket.gethostname())`.
 - Create a raw socket using `socket.socket()` with appropriate parameters.
2. **Configure Socket**
 - Bind the socket to the host IP.
 - Set socket options to include IP headers.
 - Enable promiscuous mode to capture all packets.
3. **Capture Packets**
 - Use an infinite loop to continuously receive packets using `recvfrom()`.
4. **Parse Ethernet Frame**
 - Extract the first 14 bytes of the packet for Ethernet header.
 - Unpack destination MAC, source MAC, and protocol using `struct.unpack()`.
5. **Format and Display**
 - Convert raw MAC addresses to human-readable format.
 - Convert protocol bytes to hexadecimal string.
 - Print the extracted details.

Output:

```
C:\Users\Tcs\Desktop>python3 1.py
IP: 172.16.10.79

Ethernet Frame:
Destination MAC: 45:00:04:68:D0:8D
Source MAC: 40:00:80:11:00:00
Protocol: AC10

Ethernet Frame:
Destination MAC: 45:00:00:38:00:00
Source MAC: 40:00:3C:11:2B:4D
Protocol: 8EFA

Ethernet Frame:
Destination MAC: 45:00:00:CB:B0:D8
Source MAC: 00:00:01:11:71:2E
Protocol: AC10

Ethernet Frame:
Destination MAC: 45:00:00:4A:B0:D9
Source MAC: 00:00:01:11:71:AE
Protocol: AC10
```

Code:

```
import socket
import struct
import binascii
import textwrap

def main():
    # Get host
    host = socket.gethostbyname(socket.gethostname())
    print('IP: {}'.format(host))

    # Create a raw socket and bind it
    conn = socket.socket(socket.AF_INET, socket.SOCK_RAW,
socket.IPPROTO_IP)
    conn.bind((host, 0))

    # Include IP headers
    conn.setsockopt(socket.IPPROTO_IP, socket.IP_HDRINCL, 1)
    # Enable promiscuous mode
    conn.ioctl(socket.SIO_RCVALL, socket.RCVALL_ON)

    while True:
        # Recive data
        raw_data, addr = conn.recvfrom(65536)
```

```
# Unpack data
dest_mac, src_mac, eth_proto, data = ethernet_frame(raw_data)

print('\nEthernet Frame:')
print("Destination MAC: {}".format(dest_mac))
print("Source MAC: {}".format(src_mac))
print("Protocol: {}".format(eth_proto))

# Unpack ethernet frame
def ethernet_frame(data):
    dest_mac, src_mac, proto = struct.unpack('!6s6s2s', data[:14])
    return get_mac_addr(dest_mac), get_mac_addr(src_mac),
    get_protocol(proto), data[14:]

# Return formatted MAC address AA:BB:CC:DD:EE:FF
def get_mac_addr(bytes_addr):
    bytes_str = map('{:02x}'.format, bytes_addr)
    mac_address = ':'.join(bytes_str).upper()
    return mac_address

# Return formatted protocol ABCD
def get_protocol(bytes_proto):
    bytes_str = map('{:02x}'.format, bytes_proto)
    protocol = ''.join(bytes_str).upper()
    return protocol
main()
```

RESULT

- The program successfully captures and displays Ethernet frame details for incoming packets.
- Output includes:
 - **Destination MAC Address**
 - **Source MAC Address**
 - **Protocol Type**